CORVEX / LABS

ENGINEERING IMPLEMENTATION PLAN · INDUSTRIAL AI THREAT DETECTION

From idea to a working system.

A practical build plan for a passive OT monitoring prototype that detects controlled anomalies, correlates network events with physical sensors, and preserves explainable evidence.

VERSION 0.1 | 03 AUG 2026 | TEAM OF 3

01 / MVP DEFINITION

Build one convincing industrial experiment before building a platform.

The first version is a safe lab system, not a production security product. Keep the scope narrow enough that every result can be reproduced and explained.

IN SCOPE

Local passive deployment

Laptop or Raspberry Pi, SPAN/TAP traffic, Modbus TCP first, SQLite, FastAPI and a local dashboard.

IN SCOPE

Five controlled scenarios

Unauthorized write, replay, rogue device, abnormal IT-to-OT relationship and physical divergence.

OUT OF SCOPE

No active intervention

No scanning, packet injection, automatic blocking, autonomous response or live-plant deployment.

DONE MEANS

Evidence, not screenshots

Alerts appear within five seconds, link network and sensor events, work offline and reproduce three times.

Core principle

Rules create trust. Models expand coverage. Physical sensing creates an independent evidence stream.

02 / SYSTEM ARCHITECTURE

Four layers, one local evidence loop.

CORVEX CONTROL PLANE

FastAPI service, alert triage, asset inventory, evidence report and optional fleet view. Cloud is not required for core detection.

CORVEX EDGE

Suricata/Zeek JSON ingestion, Modbus parser, normalizer, rule engine, baseline, anomaly model and local SQLite storage.

PHYSICAL WITNESS

ESP32, current clamp, vibration sensor and temperature probe. Sensor events carry asset ID, timestamp, quality and signature.

CORVEX VAULT

Append-oriented evidence records link alert, packet context, sensor readings, detector version and operator disposition.

SAFETY BOUNDARY

Read-only mirrored traffic. No control-loop writes. Attack scripts run only on an isolated lab network.

Keep the codebase understandable to three builders.

```
corvex/
├── README.md
├── pyproject.toml
├── docker-compose.yml
├── apps/
│   ├── edge_agent/
│   │   ├── capture/        suricata_reader.py, zeek_reader.py
│   │   ├── parsers/        modbus_parser.py, s7_parser.py
│   │   ├── normalize/      network_events.py, sensor_events.py
│   │   ├── detection/      rule_engine.py, baseline.py, anomaly_model.py
│   │   ├── correlation/    physical_divergence.py
│   │   └── storage/        database.py, repositories.py
│   ├── api/                FastAPI, schemas, routes
│   └── dashboard/          index.html, app.js, styles.css
├── firmware/esp32_sensor/  main.cpp, sensors.cpp
├── simulator/              normal_process.py + attack scripts
├── configs/                assets.yml, detection_rules.yml
├── tests/                  rules, baseline, correlation
└── docs/                   threat-model, test-cases, demo-script
```

MEMBER 01

Network + detection

Capture, protocol parsing, event normalization, rules, baselines and attack simulator.

MEMBER 02

Embedded + sensing

ESP32 firmware, sensor calibration, MQTT, buffering and physical testbed.

MEMBER 03

Backend + product

FastAPI, SQLite, dashboard, evidence export, demo script and documentation.

SHARED

End-to-end review

All three members understand the complete demo. Avoid three disconnected mini-projects.

04 / TECHNOLOGY CHOICES

Use existing engines. Spend your time on the evidence loop.

LAYER	CHOICE	REASON
Runtime	Python 3.11+	Fast iteration and strong data/ML ecosystem.
Telemetry	Suricata EVE JSON and/or Zeek JSON	Structured events without writing a packet engine.
Protocol	Modbus TCP first	Easy to simulate and demonstrate.
API	FastAPI + Pydantic	Typed schemas and fast local service.
Storage	SQLite + SQLAlchemy	Offline and simple for MVP.
Detection	Rules, rolling statistics, Isolation Forest later	Explainable first, ML after clean baseline data.
Firmware	ESP32 + PlatformIO	Reproducible embedded builds.
Messaging	MQTT with TLS	Sensor events and later fleet replication.
Cloud	None required for v0.1	Core product must work without internet.

Suricata documents EVE JSON output for alerts and protocol records. Zeek documents structured JSON log output. Use their output as inputs, not as your product boundary.

Normalize once. Let every detector share the same truth.

```
{
  "event_id": "evt-000001",
  "timestamp": "2026-08-03T16:00:00.123Z",
  "source": "suricata",
  "asset_id": "plc-01",
  "src_ip": "192.168.10.20",
  "dst_ip": "192.168.10.30",
  "protocol": "modbus",
  "function_code": 3,
  "register": 40001,
  "value": 40,
  "event_type": "read",
  "raw_ref": "suricata-line-8841"
}
```

```
{
  "event_id": "sensor-000001",
  "timestamp": "2026-08-03T16:00:00.140Z",
  "source": "esp32-01",
  "asset_id": "motor-01",
  "measurements": { "current_amp": 12.4, "vibration_rms": 0.18 },
  "quality": "good",
  "signature": "sha256:..."
}
```

Detectors should never understand raw Suricata, Zeek and firmware formats separately. Convert each input to this internal contract, then build rules and correlation on top.

06 / DETECTION IMPLEMENTATION

Do the boring, reliable work first.

01 / SIMULATE

Normal process

Create registers 40001 motor speed, 40002 current and 40003 temperature. Generate predictable polling.

02 / CAPTURE

Normalize events

Read JSON output, parse Modbus fields and save events to SQLite.

03 / DETECT

Rules first

Unauthorized source writes, out-of-range registers, new devices, timing drift and replay patterns.

04 / CORRELATE

Reality check

Compare controller state with current or vibration and create a linked evidence record.

```
def unauthorized_write(event, allowed_writers):
    if event.get("protocol") != "modbus": return None
    if event.get("event_type") != "write": return None
    if event.get("src_ip") in allowed_writers: return None
    return {"alert_type":"UNAUTHORIZED_WRITE",
            "severity":"high",
            "asset_id":event["asset_id"],
            "reason":"Unapproved device wrote to a control register"}
```

AI comes after clean data and a clear explanation.

BASELINE FEATURES

What to store

Median polling interval, interval range, function codes, allowed register ranges, write frequency, normal peers and sensor statistics.

MODEL PROCESS

How to validate

Collect normal traffic, build feature windows, train Isolation Forest offline, save model version and measure false positives on a separate window.

```
SCADA claims:     motor_speed = 40 Hz
Network appears:  valid Modbus response
Sensor measures:   current = 19.1 A
                   vibration = +240%
CORVEX result:     PHYSICAL_DIVERGENCE
```

For v0.1, use a calibrated relationship between motor speed and current. Treat the threshold as a lab placeholder until measurements validate it. Physical divergence can also mean mechanical or sensor fault, so the alert is a hypothesis requiring operator review.

Build only the screens that prove the system works.

```
GET  /health
GET  /assets
GET  /assets/{asset_id}
GET  /alerts?status=open
GET  /alerts/{alert_id}
GET  /alerts/{alert_id}/evidence
POST /alerts/{alert_id}/acknowledge
GET  /baseline/{asset_id}
GET  /reports/{alert_id}
```

SCREEN 01

Asset inventory

Asset ID, IP, protocol, peers, first seen and last seen.

SCREEN 02

Open alerts

Severity, type, asset, timestamp and status.

SCREEN 03

Alert evidence

Reason, packet context, sensor context and detector version.

SCREEN 04

Baseline status

Training window, coverage, drift and model version.

09 / SEVEN-DAY SPRINT

What the team should do next.

DAY	DELIVERABLE	DEFINITION OF DONE
01	Modbus simulator	Normal motor traffic and three registers work repeatably.
02	Traffic capture	Zeek or Suricata produces readable JSON events.
03	Event schema	Network events are stored in SQLite using one contract.
04	First rule	Unauthorized Modbus write creates an explainable alert.
05	Dashboard	Asset list and alert detail page work locally.
06	One sensor	ESP32 publishes current or vibration data with timestamps.
07	End-to-end demo	Physical divergence produces a linked evidence record.

After the first demo

Add the other four simulators, baseline lock, replay detection, report export, automated tests and a three-minute recording.

Do not call it finished until it can prove these things.

COVERAGE

Three assets, five attacks

All scenarios are reproducible from clean setup.

SAFETY

No process interference

No packet injection and no control-loop writes.

EVIDENCE

Linked context

Every alert links network event, sensor event, timestamp and reason.

OFFLINE

Local-first

Detection works with internet disconnected.

SECURITY

Basic hygiene

No default passwords, pinned dependencies, environment secrets and TLS for external MQTT.

HONESTY

Limitations labelled

Simulator data, calibration limits and prototype status are visible in the demo.

References: Zeek logs, https://docs.zeek.org/en/v8.1.0/log-formats.html · Suricata EVE JSON, https://docs.suricata.io/en/latest/output/eve/eve-json-output.html · AWS IoT Raspberry Pi guide, https://docs.aws.amazon.com/iot/latest/developerguide/connecting-to-existing-device.html

FINAL ENGINEERING PRINCIPLE

Do not build a dashboard. Build evidence.

A safe industrial experiment with a clear result beats a large architecture diagram with no working proof.